

CASE STUDY – BFS, DFS, Prim's and Kruskal's Algorithms

CO3: Apply BFS and DFS graph traversal algorithms and construct Minimum Spanning Trees using Kruskal's and Prim's algorithms to solve infrastructure design, network connectivity, and cluster analysis problems in weighted undirected graphs.

Scenario / Case Study:

A company named NetConnect Solutions is designing a communication network between offices. The offices are represented as graph vertices and communication links are represented as weighted edges. The company must: Traverse the network using BFS and DFS Find the Minimum Spanning Tree (MST) Reduce installation cost using Prim's and Kruskal's algorithms Graph Vertices: A, B, C, D, E Weighted Edges: (A-B=2), (A-C=3), (B-C=1), (B-D=4), (C-D=5), (C-E=6), (D-E=7)

(a) BFS and DFS Traversal

Starting vertex: A Perform BFS and DFS traversal for the given graph.

BFS Traversal:

A → B → C → D → E

DFS Traversal:

A → B → C → D → E

Observation: BFS explores level-by-level, while DFS explores depth-wise. Both traversals are useful in network connectivity and path discovery.

(b) Sub Question – Java Program for BFS Traversal

Write a Java program to perform Breadth First Search (BFS) traversal on a graph.

```
import java.util.*;

class Graph {
    private int V;
    private LinkedList<Integer> adj[];

    Graph(int v) {
        V = v;
        adj = new LinkedList[V];

        for (int i = 0; i < v; ++i)
            adj[i] = new LinkedList();
    }

    void addEdge(int v, int w) {
        adj[v].add(w);
    }

    void BFS(int s) {
        boolean visited[] = new boolean[V];

        LinkedList<Integer> queue = new LinkedList<Integer>();

        visited[s] = true;
        queue.add(s);

        while (queue.size() != 0) {
            s = queue.poll();

            System.out.print(s + " ");
            Iterator<Integer> i = adj[s].listIterator();
```

```

        while (i.hasNext()) {
            int n = i.next();

            if (!visited[n]) {
                visited[n] = true;
                queue.add(n);
            }
        }
    }

    public static void main(String args[]) {
        Graph g = new Graph(5);

        g.addEdge(0, 1);
        g.addEdge(0, 2);
        g.addEdge(1, 3);
        g.addEdge(2, 4);

        System.out.println("BFS Traversal:");

        g.BFS(0);
    }
}

```

OUTPUT:

```

BFS Traversal:
0 1 2 3 4

```

(c) Prim's Algorithm for MST

Construct the Minimum Spanning Tree using Prim's Algorithm.

Selected Edges:

```

(B-C = 1)
(A-B = 2)
(B-D = 4)
(C-E = 6)

```

Total Minimum Cost = 13

(d) Kruskal's Algorithm for MST

Construct the Minimum Spanning Tree using Kruskal's Algorithm.

Edges Sorted by Weight:

```

(B-C=1), (A-B=2), (A-C=3), (B-D=4), (C-D=5), (C-E=6), (D-E=7)

```

Selected MST Edges:

```

(B-C=1)
(A-B=2)
(B-D=4)
(C-E=6)

```

Total Minimum Cost = 13

Comparison of Algorithms

Algorithm	Purpose	Complexity
BFS	Level-wise Traversal	$O(V + E)$
DFS	Depth-wise Traversal	$O(V + E)$

Prim's Algorithm	Minimum Spanning Tree	$O(E \log V)$
Kruskal's Algorithm	Minimum Spanning Tree	$O(E \log E)$

Conclusion

BFS and DFS are essential graph traversal techniques used in connectivity analysis and routing. Prim's and Kruskal's algorithms efficiently construct Minimum Spanning Trees for cost optimization in networks. These algorithms are widely used in communication systems, infrastructure planning, and clustering applications.